



第五部分 TCP协议



第五部分.1 TCP协议介绍



TCP简介

UDP是一种没有复杂控制，提供面向无连接通信服务的一种协议。换句话说，它将部分控制转移给应用程序去处理（例如TFTP），自己却只提供作为传输层协议的最基本功能。

与UDP不同，TCP则“人如其名”，可以说是对“传输、发送、通信”进行“控制”的“协议”。

与UDP的区别相当大。它充分地实现了数据传输时各种控制功能，可以进行丢包时的重发控制，还可以对次序乱掉的分包进行顺序控制。而这些在UDP中都没有。此外，TCP作为一种面向有连接的协议，只有在确认通信对端存在时才会发送数据，从而可以控制通信流量的浪费。

根据TCP的这些机制，在IP这种无连接的网络上也能够实现高可靠性的通信。



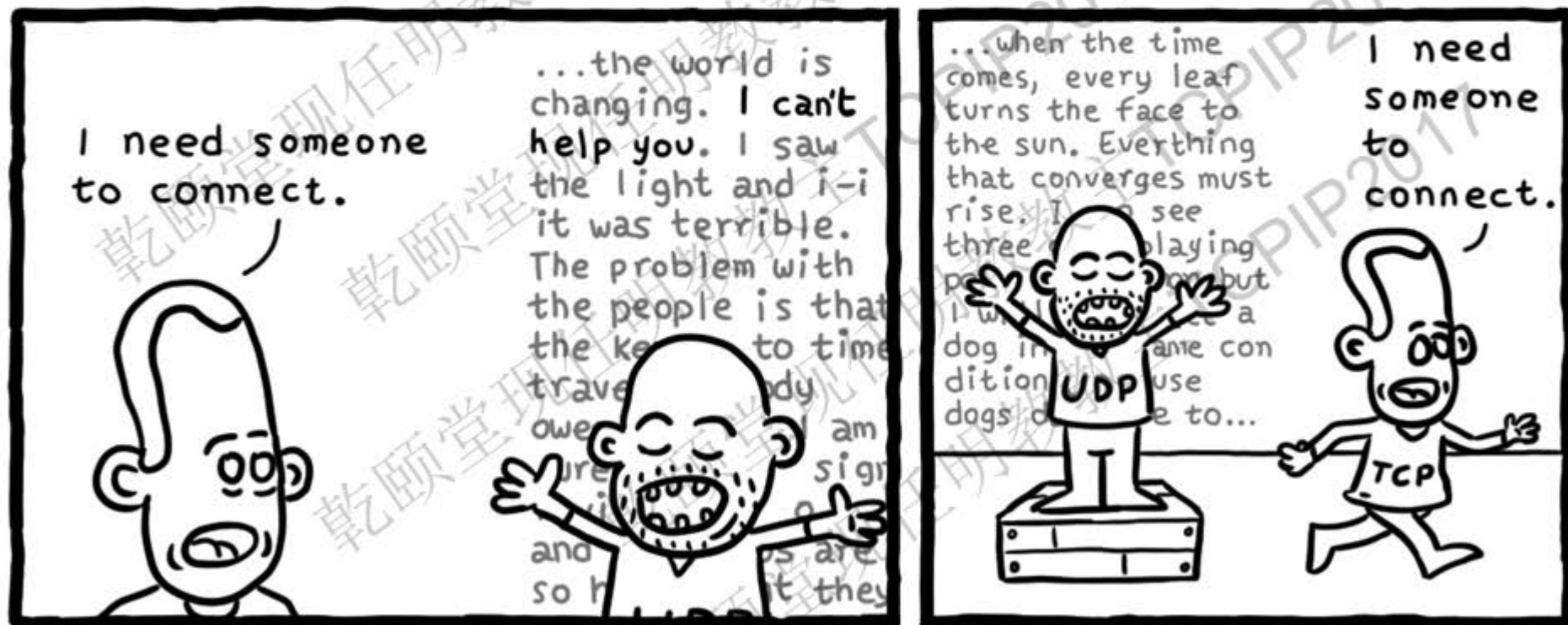
1.连接的管理

TCP提供面向有连接的通信传输。面向有连接是指在数据通信开始之前先做好通信两端之间的准备工作。

UDP是一种面向无连接的通信协议，因此不检查对端是否可以通信，直接将UDP包发送出去。**TCP**与此相反，它会在数据通信之前，通过**TCP**首部发送一个**SYN**包作为建立连接的请求等待确认应答。如果对端发来确认应答，则认为可以进行数据通信。如果对端的确认应答未能到达，就不会进行数据通信。此外，在通信结束时将进行断开连接的处理（**FIN**包）。

可以使用**TCP**首部用于控制的字段来管理**TCP**连接，一个连接的建立与断开，正常过程至少需要来回发送7个包才能完成。

TCP vs UDP漫画



三次握手建立连接





TCP建立与结束会话的7个包





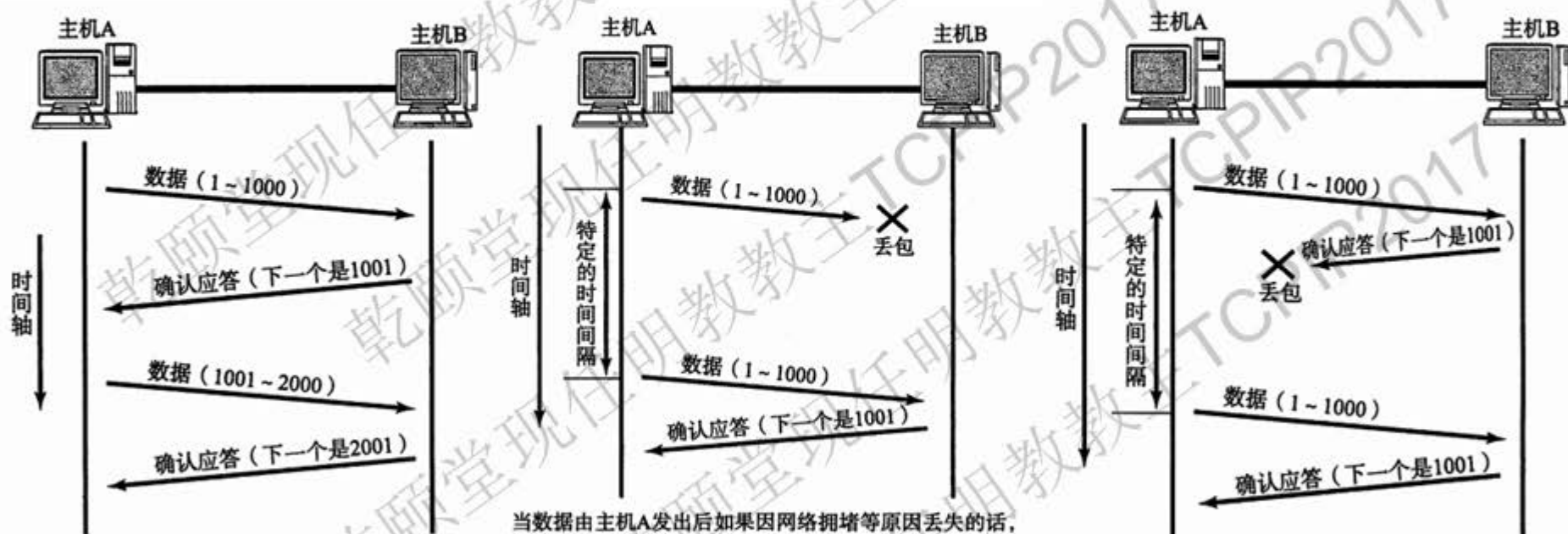
2.通过序列号与确认应答提高可靠性

在TCP中，当发送端的数据到达接收主机时，接收端主机会返回一个已收到消息的通知。**这个消息叫做确认应答(ACK)**。

通常，两个人对话时，在谈话的停顿处可以点头或询问以确认谈话内容。**如果对方迟迟没有任何反馈，说话的一方还可以再重复一遍以保证对方确实听到**。因此，对方是否理解了此次对话内容，对方是否完全听到了对话的内容，都要靠对方的反应来判断。网络中的“确认应答”就是类似这样的一个概念。当对方听懂对话内容时会说：“**嗯**”，这就相当于返回了一个确认应答(ACK)。

第五部分.1:TCP协议介绍

TCP ACK确认工作示意图



当数据从主机A发送到主机B时，主机B会返回给主机A一个确认应答。

当数据由主机A发出后如果因网络拥堵等原因丢失的话，该数据将无法到达主机B。此时，如果主机A在一个特定时间间隔内都未收到主机B发来的确认应答，将会对此数据进行重发。

由主机B返回的确认应答，因网络拥堵等原因在传送的途中丢失，没有到达主机A。主机A会等待一段时间，若在特定的时间间隔内始终未能收到这个确认应答，主机A会对此数据进行重发。此时，主机B将第二次发送已接收此数据的确认应答。由于主机B其实已经收到过1~1000的数据，当再有相同数据送达时它会放弃。

任何一个方向的丢包对于发送端而言并没有什么区别
关于重复接收相同数据包问题，由即将介绍的序列号来解决



3. 序列号简介

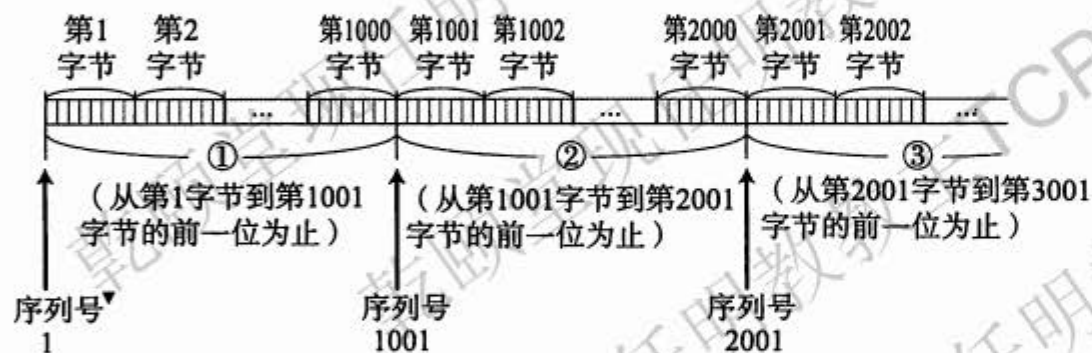
可能因为一些其他原因导致确认应答延迟到达，在源主机重发数据以后才到达的情况也屡见不鲜。此时，源发送主机只要按照机制重发数据即可。但是对于目标主机来说，这简直是一种“灾难”。它会**反复收到相同的数据**。而为了对上层应用提供可靠的传输，**必须得放弃重复的数据包**。为此，就必须引入一种机制，它能够识别是否已经接收数据，又能够判断是否需要接收。

上述这些确认应答处理、重发控制以及重复控制等功能都可以通过序列号实现。**序列号是按顺序给发送数据的每一个字节（8位字节）都标上号码的编号**。接收端查询接收数据TCP首部中的序列号和数据的长度，**将自己下一步应该接收的序号作为确认应答返送回去**。就这样，通过序列号和确认应答号，TCP可以实现可靠传输。

第五部分.1:TCP协议介绍

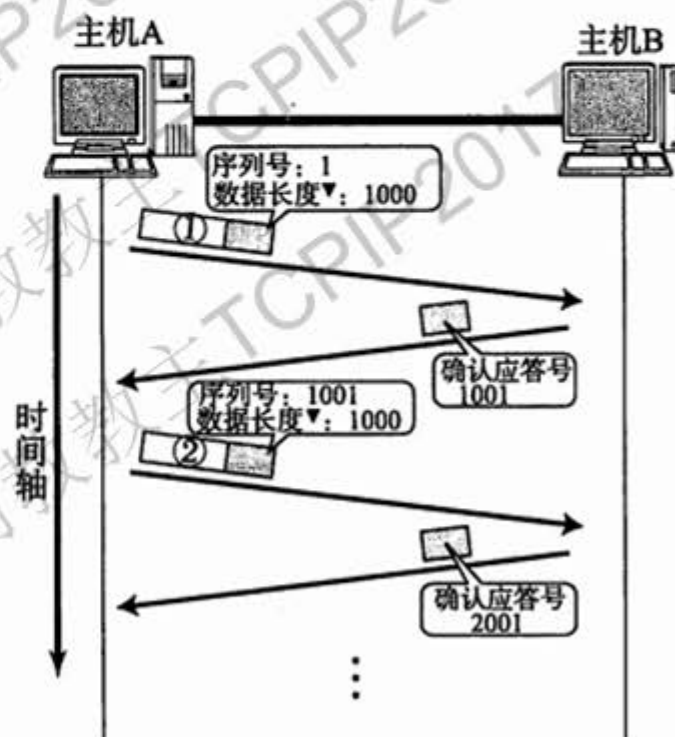
序列号示意图

· 发送的数据



返回下一个
希望得到字
节的序列号

· 序列号与确认应答号





4. TCP以段为单位发送数据

在**建立TCP连接**的同时，也可以确定发送数据包的单位，我们也可以称其为“最大消息长度”（MSS: Maximum Segment Size）。最理想的情况是，最大消息长度正好是IP中不会被分片处理的最大数据长度（一般为 $1460 = 1500 - 20 - 20$ ）。

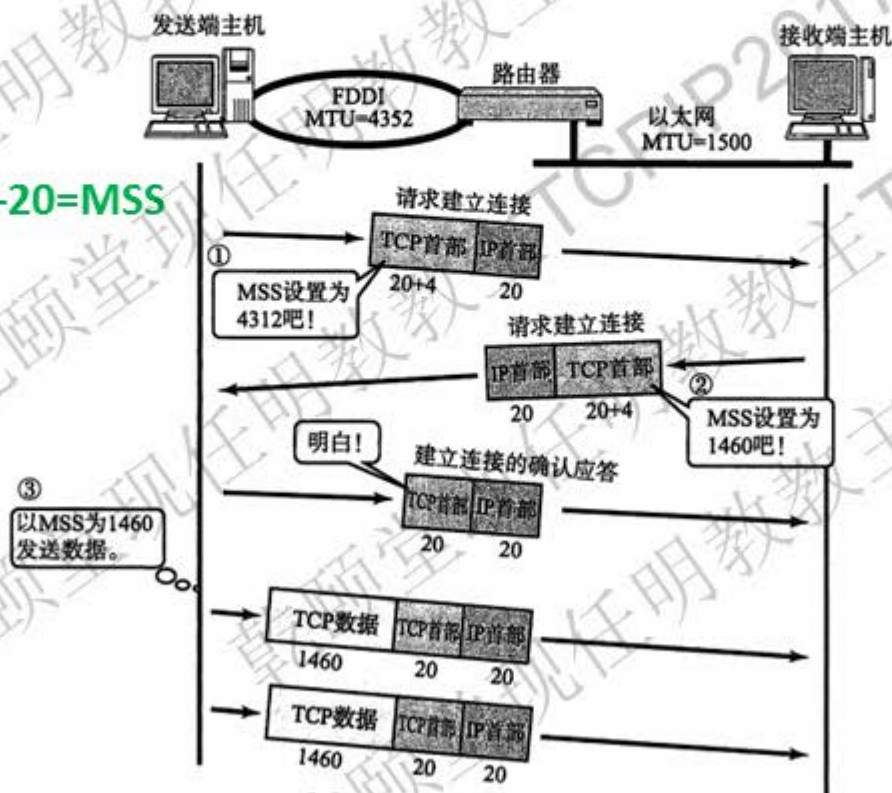
TCP在传送大量数据时，是以MSS的大小将数据进行分割发送。进行重发时也是以MSS为单位。

MSS是在三次握手的时候，在两端主机之间被计算得出。两端的主机在发出建立连接的请求时，会在TCP首部中写入MSS选项，告诉对方自己的接口能够适应的MSS的大小。然后会在两者之间选择一个较小的值投入使用。

第五部分.1:TCP协议介绍

MSS协商示意图

发起端的MTU-20-20=MSS



(图中数字表示数据的长度, 单位为8位字节。
确认应答的段有一部分已省略。)

①② 通过建立连接的SYN包相互通知对方网络接口的MSS值。

③ 在两者之间选一个较小的作为MSS的值, 发送数据。



第五部分.1:TCP协议介绍

ip tcp adjust-mss

TCP - Transport Control Protocol

Source Port: 42277 [34-35]
Destination Port: 23 telnet [36-37]
Sequence Number: 2241244997 [38-41]
Ack Number: 0 [42-45]
TCP Offset: 6 (24 bytes) [46 Mask 0xF0]
Reserved: %0000 [46 Mask 0x0F]
TCP Flags: %00000010S. [47]
0... .. (No Congestion Window Reduction)
.0... .. (No ECN-Echo)
..0... .. (No Urgent pointer)
...0... .. (No ACK)
....0... .. (No Push)
.....0... .. (No Reset)
.....1... SYN
.....0... (No FIN)
Window: 4128 [48-49]
TCP Checksum: 0x3665 [50-51]
Urgent Pointer: 0 [52-53]

TCP Options:

Option Type: 2 Maximum Segment Size [54]
Length: 4 [55]
MSS: 536 [56-57]

TCP - Transport Control Protocol

Source Port: 23 telnet [34-35]
Destination Port: 42277 [36-37]
Sequence Number: 579828056 [38-41]
Ack Number: 2241244998 [42-45]
TCP Offset: 6 (24 bytes) [46 Mask 0xF0]
Reserved: %0000 [46 Mask 0x0F]
TCP Flags: %00010010 ...A..S. [47]
0... .. (No Congestion Window Reduction)
.0... .. (No ECN-Echo)
..0... .. (No Urgent pointer)
...1... ACK
....0... .. (No Push)
.....0... .. (No Reset)
.....1... SYN
.....0... (No FIN)
Window: 4128 [48-49]
TCP Checksum: 0x9A90 [50-51]
Urgent Pointer: 0 [52-53]

TCP Options:

Option Type: 2 Maximum Segment Size [54]
Length: 4 [55]
MSS: 500 [56-57]

Extra bytes

Number of bytes: (2 bytes) [58-59]

R2:

interface GigabitEthernet2

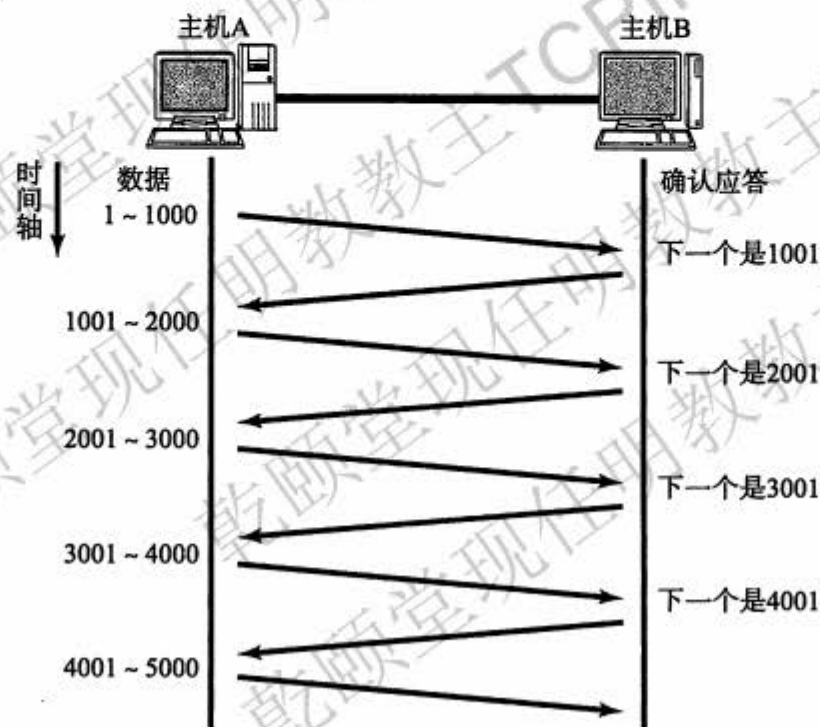
ip address 172.16.23.2 255.255.255.0

ip tcp adjust-mss 500

5. 滑动窗口技术简介（1）

TCP以一个段为单位，每发一个段进行一次确认应答的处理，如下图所示。这样的传输方式有一个缺点。那就是，包的往返时间越长通信性能就越低。

TFTP工作原理就
如此，停止等待

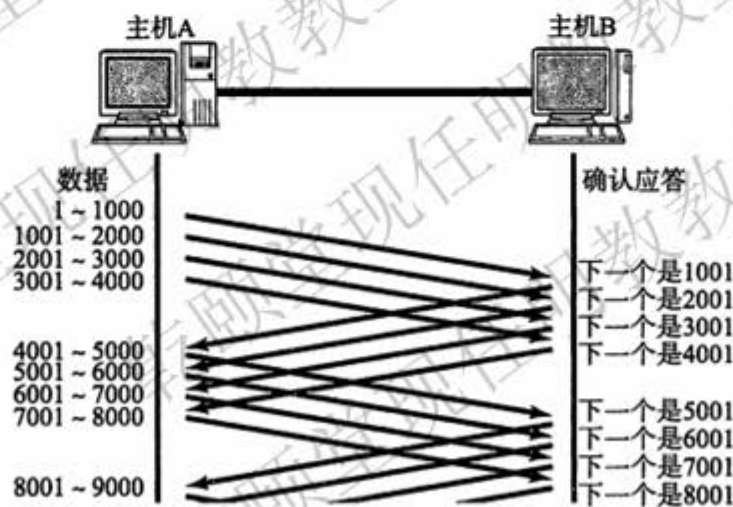


为每个数据包进行确认应答的缺点是，包的往返时间越长，网络的吞吐量会越差。

5. 滑动窗口技术简介（2）

为解决这个问题，TCP引入了窗口这个概念。即使在往返时间较长的情况下，它也能控制网络性能的下降。如下图所示，确认应答不再是以每一个分段，而是以更大的单位进行确认时，转发时间将会被大幅度的缩短。也就是说，**发送端主机，在发送了一个段之后不必要一直等待确认应答，而是继续发送。**

窗口大小就是指无需等待确认应答而可以继续发送数据的最大值。下图中，窗口大小为4个段。



窗口大小其实也是接收方缓存大小

根据窗口为4000字节时返回的确认应答，下一步就发送比这个值还要大4000个序列号为止的数据。这跟前面每个段接收确认应答以后再发送另一个新段的情况相比，即使往返时间变长也不会影响网络的吞吐量。

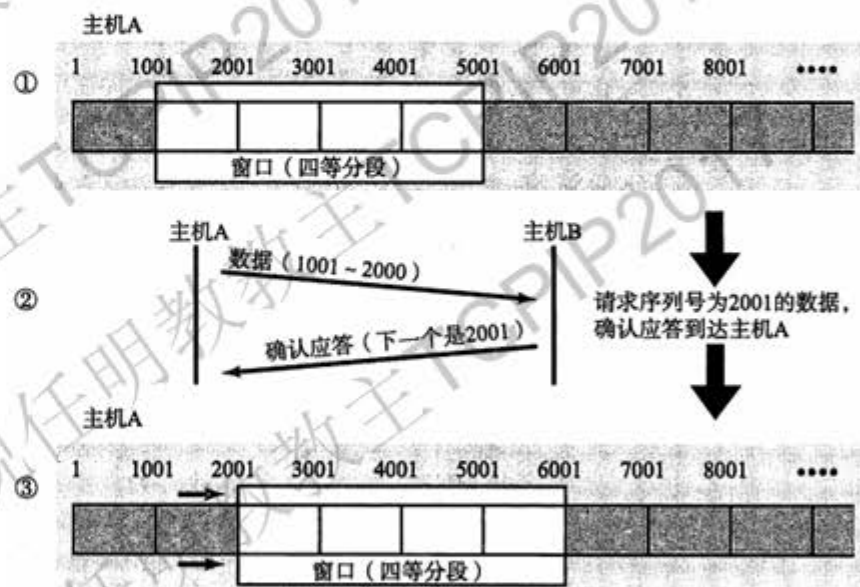
第五部分.1:TCP协议介绍

5. 滑动窗口技术简介 (3)

如右图所示, 发送数据中高亮圈起的部分正是前面所提到的窗口。在这个窗口内的数据即便没有收到确认应答也可以继续发送。此外, 从该窗口中能看到的数因其某种数据已在传输中丢失, 所以发送端无法收到确认应答, 这种情况也需进行重发。为此, 发送端主机在等到确认应答返回之前, 必须在缓冲区中保留这部分数据。

在滑动窗口以外的部分包括尚未发送的数据以及已经确认对端已收到的数据。当数据发出后若如期收到确认应答就可以不用再进行重发, 此时数据就可以从缓存区清除。

收到确认应答的情况下, 将窗口滑动到确认应答中的序列号的位置。这样可以顺序地将多个段同时发送提高通信性能。这种机制也被称为滑动窗口控制。



在①的状态下, 如果收到一个请求序列号为2001的确认应答, 那么2001之前的数据就没有必要进行重发, 这部分的数据可以被过滤掉, 滑动窗口成为③的样子。(这是在1个段为1000个字节, 窗口为4个段的情况)



第五部分.1:TCP协议介绍

窗口字段抓包

TCP - Transport Control Protocol	
Source Port:	42277 [34-35]
Destination Port:	23 telnet [36-37]
Sequence Number:	2241244997 [38-41]
Ack Number:	0 [42-45]
TCP Offset:	6 (24 bytes) [46 Mask 0xF0]
Reserved:	%0000 [46 Mask 0x0F]
TCP Flags:	%00000010S. [47]
	0... .. (No Congestion Window Reduction)
	.0... .. (No ECN-Echo)
	..0. (No Urgent pointer)
	...0 (No Ack)
	 0... (No Push)
	0.. (No Reset)
	1. SYN
	0 (No FIN)
Window:	4128 [48-49]
TCP Checksum:	0x3665 [50-51]
Urgent Pointer:	0 [52-53]
TCP Options:	
Option Type:	2 Maximum Segment Size [54]
Length:	4 [55]
MSS:	536 [56-57]
Extra bytes	
Number of bytes:	(2 bytes) [58-59]



第五部分.1:TCP协议介绍

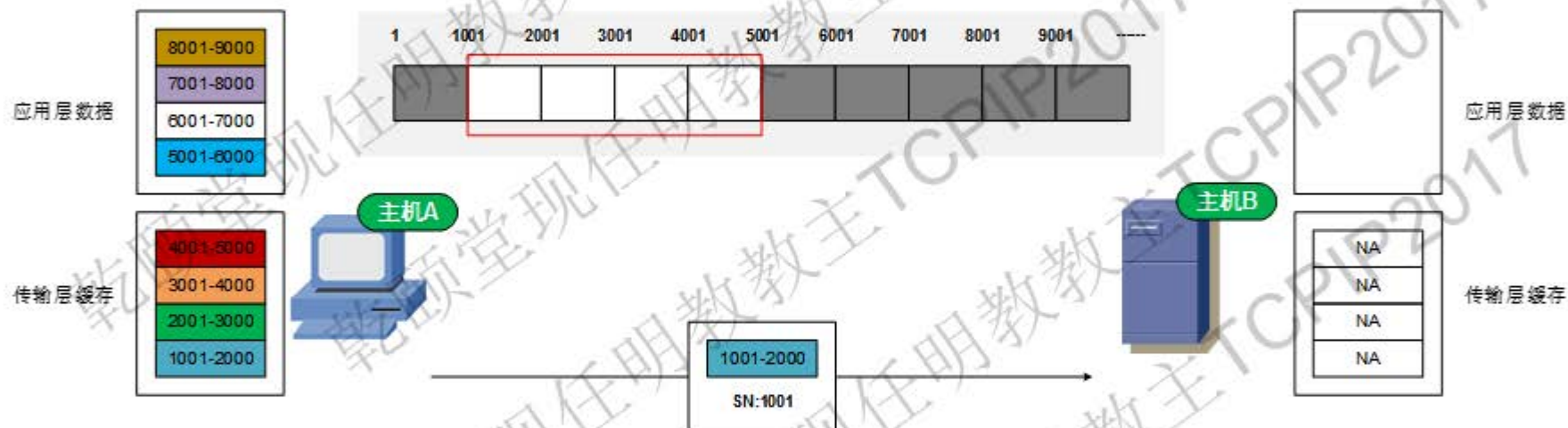
滑动窗口工作示意图 (1)





第五部分.1:TCP协议介绍

滑动窗口工作示意图 (2)





第五部分.1:TCP协议介绍

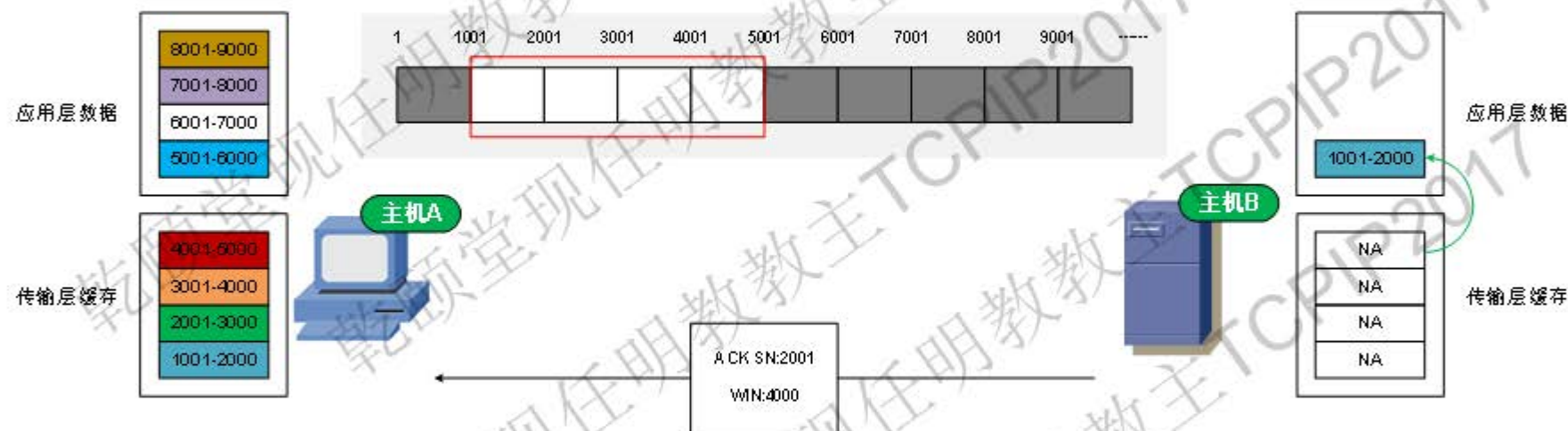
滑动窗口工作示意图 (3)





第五部分.1:TCP协议介绍

滑动窗口工作示意图 (4)





第五部分.1:TCP协议介绍

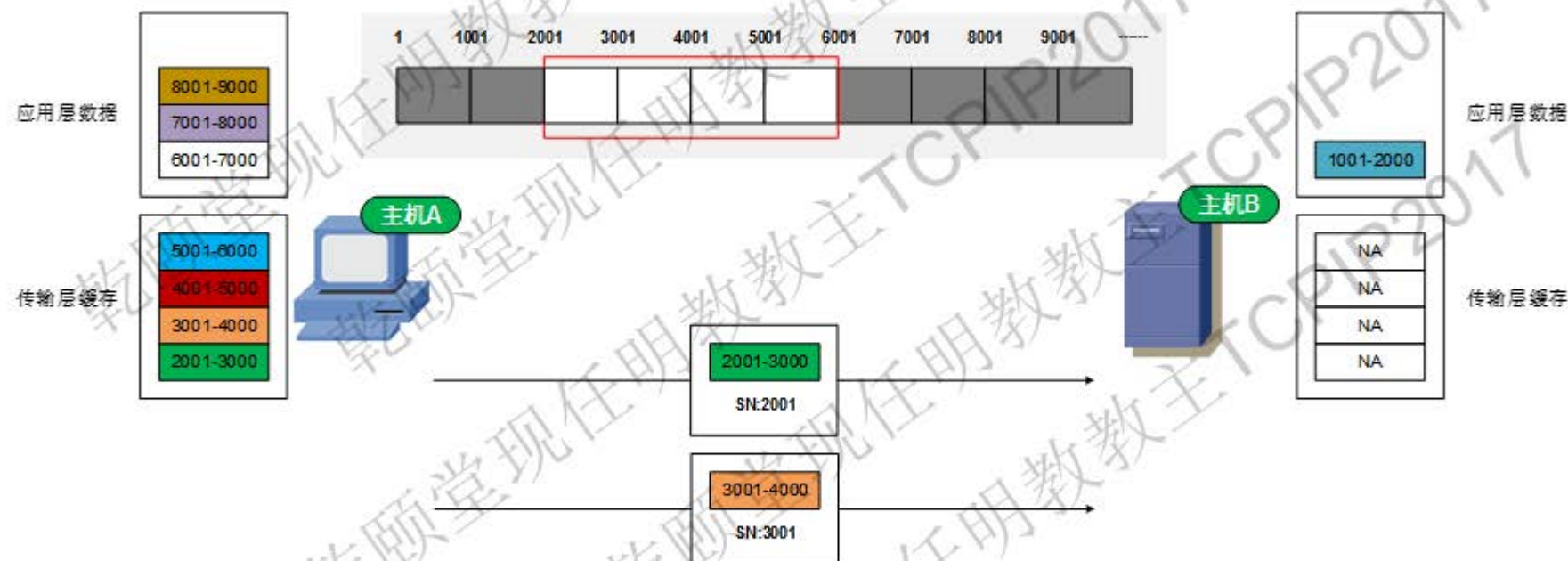
滑动窗口工作示意图 (5)





第五部分.1:TCP协议介绍

滑动窗口工作示意图 (6)





第五部分.1:TCP协议介绍

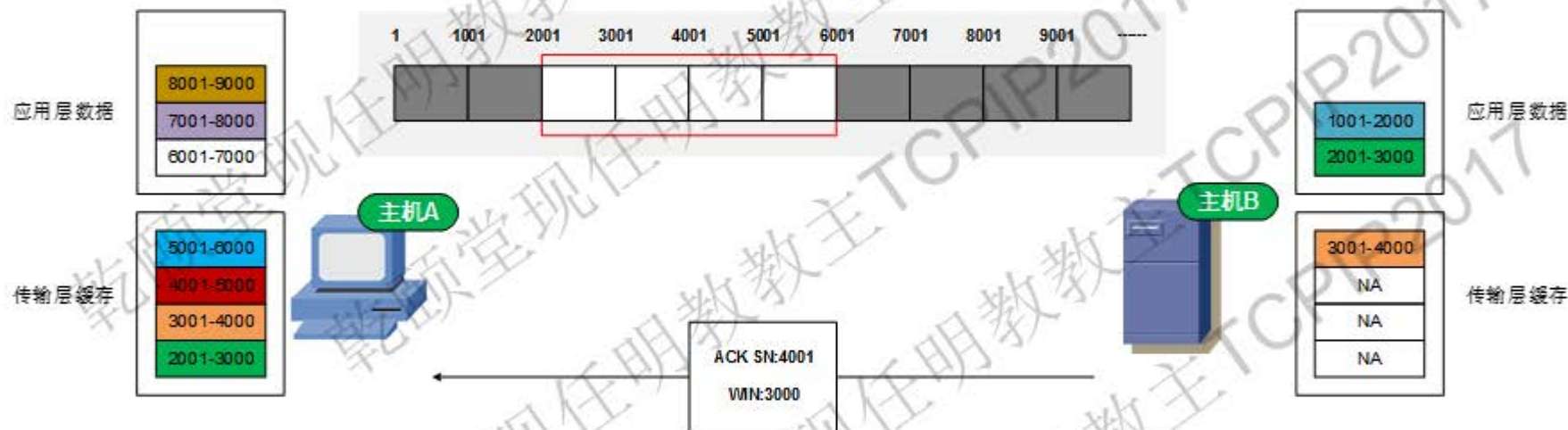
滑动窗口工作示意图 (7)





第五部分.1:TCP协议介绍

滑动窗口工作示意图 (8)



SACK选项类型，窗口
反应发送ACK当时的
接受方可用缓存情况



第五部分.1:TCP协议介绍

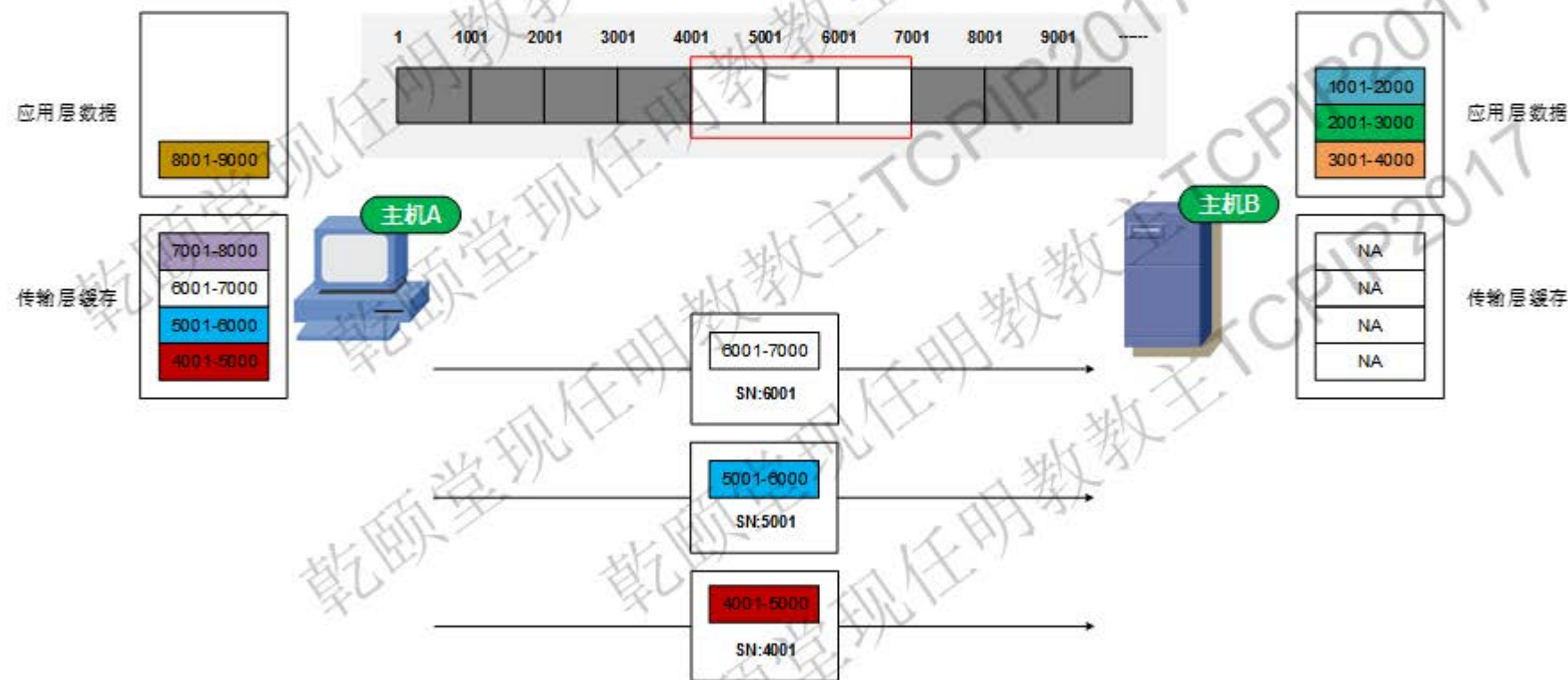
滑动窗口工作示意图 (9)





第五部分.1:TCP协议介绍

滑动窗口工作示意图 (10)





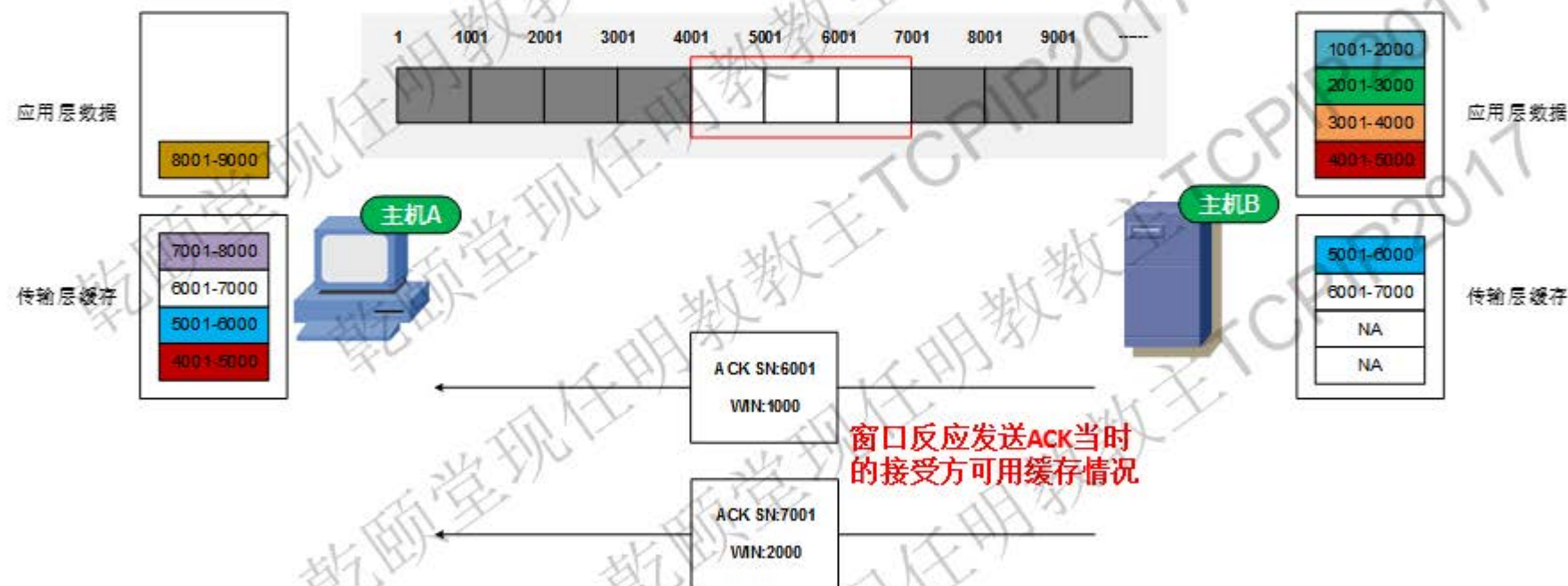
第五部分.1:TCP协议介绍

滑动窗口工作示意图 (11)



第五部分.1:TCP协议介绍

滑动窗口工作示意图 (12)





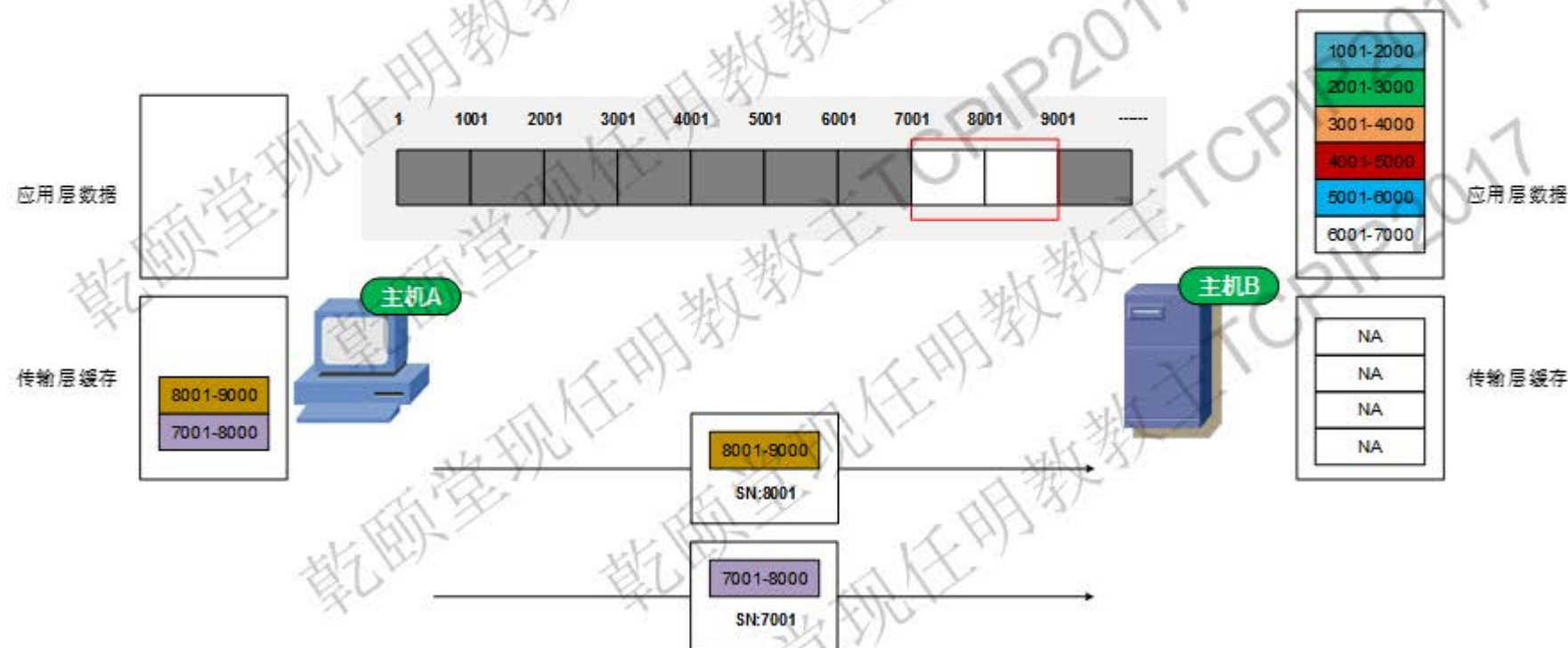
第五部分.1:TCP协议介绍

滑动窗口工作示意图 (13)



第五部分.1:TCP协议介绍

滑动窗口工作示意图 (14)



第五部分.1:TCP协议介绍

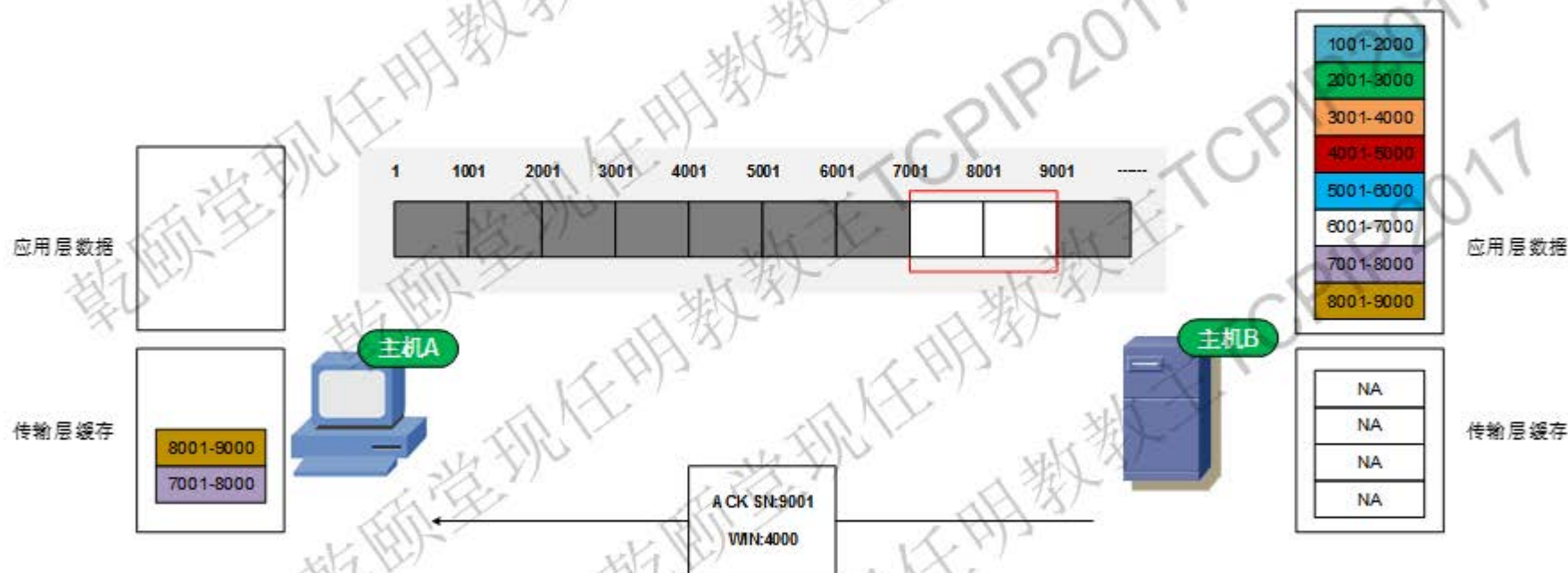
滑动窗口工作示意图 (15)





第五部分.1:TCP协议介绍

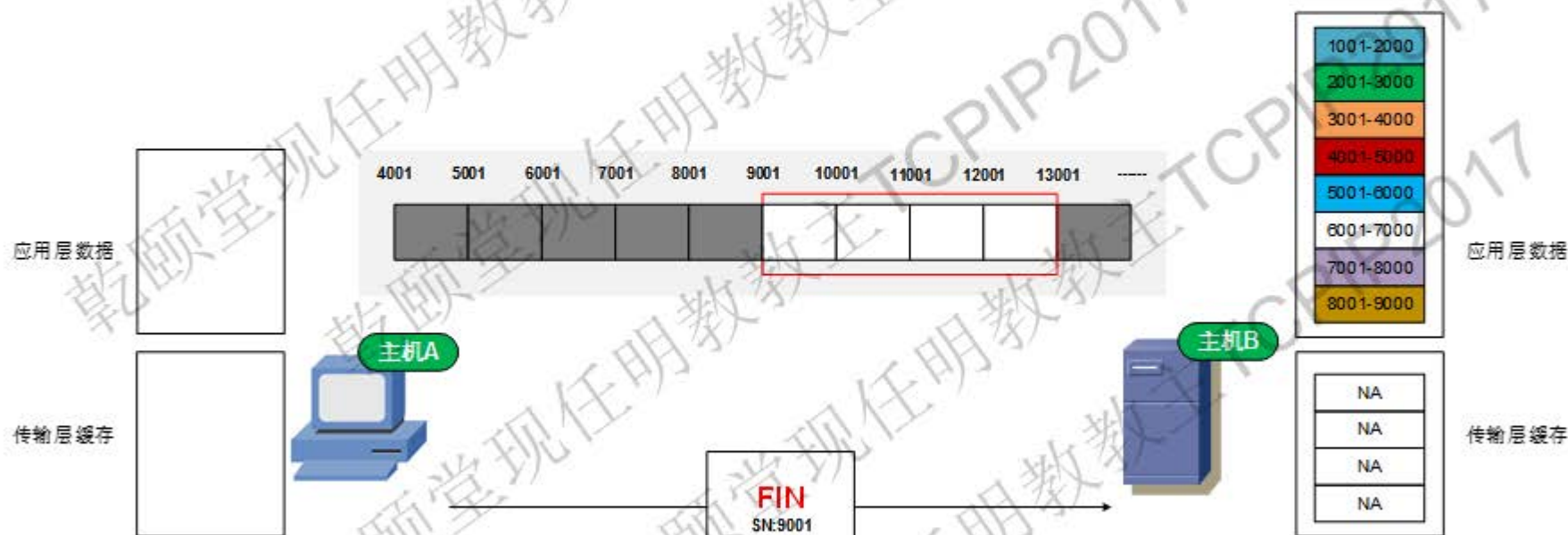
滑动窗口工作示意图 (16)





第五部分.1:TCP协议介绍

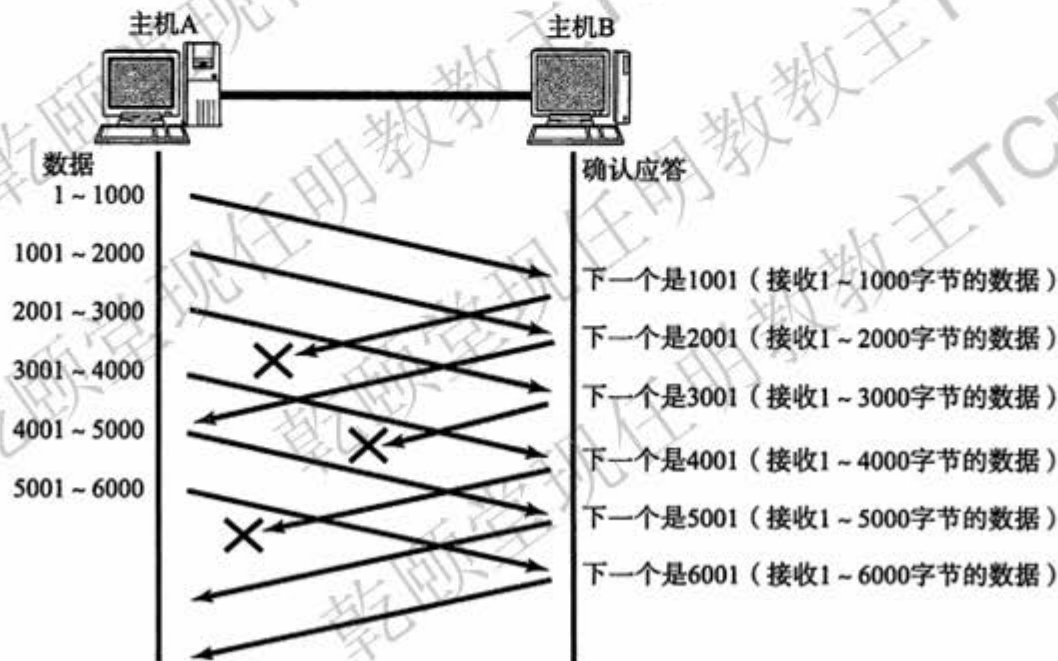
滑动窗口工作示意图 (17)



6. 窗口控制与重发控制 (1)

在使用窗口控制中，如果出现段丢失该怎么办？

首先，我们先考虑**确认应答未能返回的情况**。在这种情况下，数据已经到达对端，是不需要再进行重发的。然而，在没有使用窗口控制的时候，没有收到确认应答的数据都会被重发。而使用了窗口控制，就如下图所示，**某些确认应答即便丢失也无需重发。**

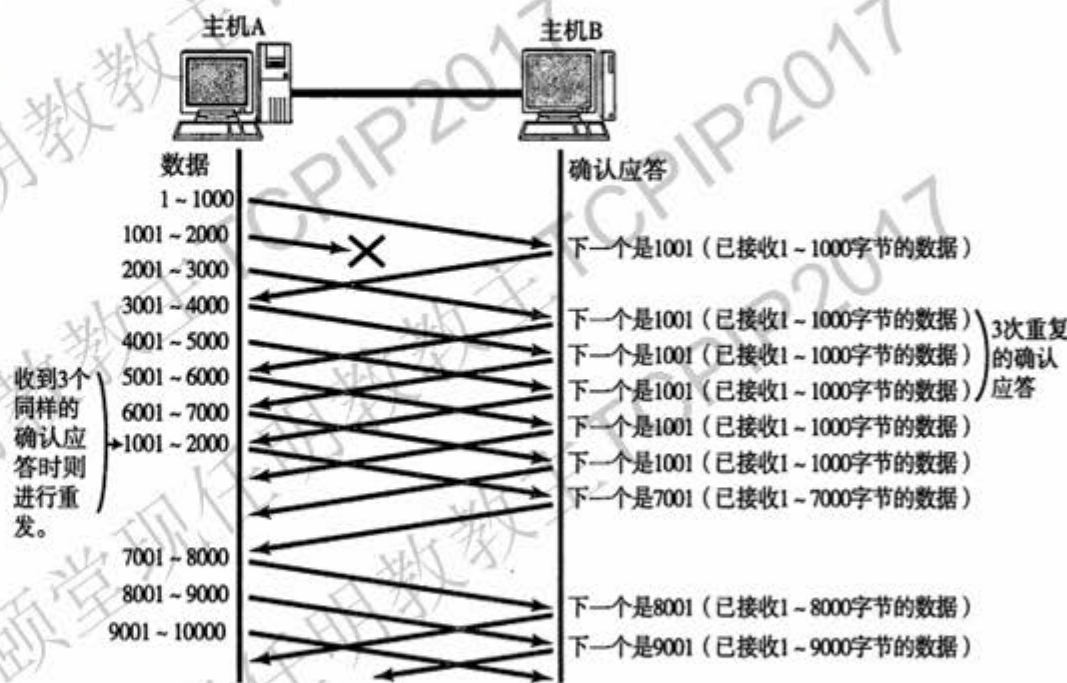


窗口在一定程度上较大时，即使有少部分的确认应答丢失也不会进行数据重发。可以通过下一个确认应答进行确认。

6. 窗口控制与重发控制 (2)

其次，我们来考虑一下某个报文段丢失的情况。如下图所示，接收主机如果收到一个自己应该接收的序号以外的数据时，会针对当前为止收到数据返回确认应答。

如下图所示。当某一报文段丢失后，发送端会一直收到序号为1001的确认应答，这个确认应答好像在提醒发送端“我想接收的是从1001开始的数据”。因此，在窗口比较大，又出现报文段丢失的情况下，同一个序号的确认应答将会被重复不断地返回。而发送端主机如果连续3次收到同一个确认应答，就会将其所对应的数据进行重发。这种机制比之前提到的超时管理更加高效，因此也被称作高速重发控制。



接收端在没有收到自己所期望序号的数据时，会对之前收到的数据进行确认应答。发送端一旦收到某个确认应答后，又连续3次收到同样的确认应答，则认为数据段已经丢失，需要进行重发。这种机制比起超时机制可以提供更为快速的重发服务。

卷一经典语句:

TCP可以表述为一个没有**选择确认**或**否认**的滑动窗口协议
SACK功能使TCP拥有了**选择确认**的能力



7. 流控制 (1)

发送端根据自己的实际情况发送数据。但是，接收端可能收到的是一个毫无关系的数据包又可能会在处理其他问题上花费一些时间。因此在为这个数据包做其他处理时会耗费一些时间，甚至在高负荷的情况下无法接收任何数据。如此一来，如果接收端将本应该接收的数据丢弃的话，就又会触发重发机制，从而导致网络流量的无端浪费。

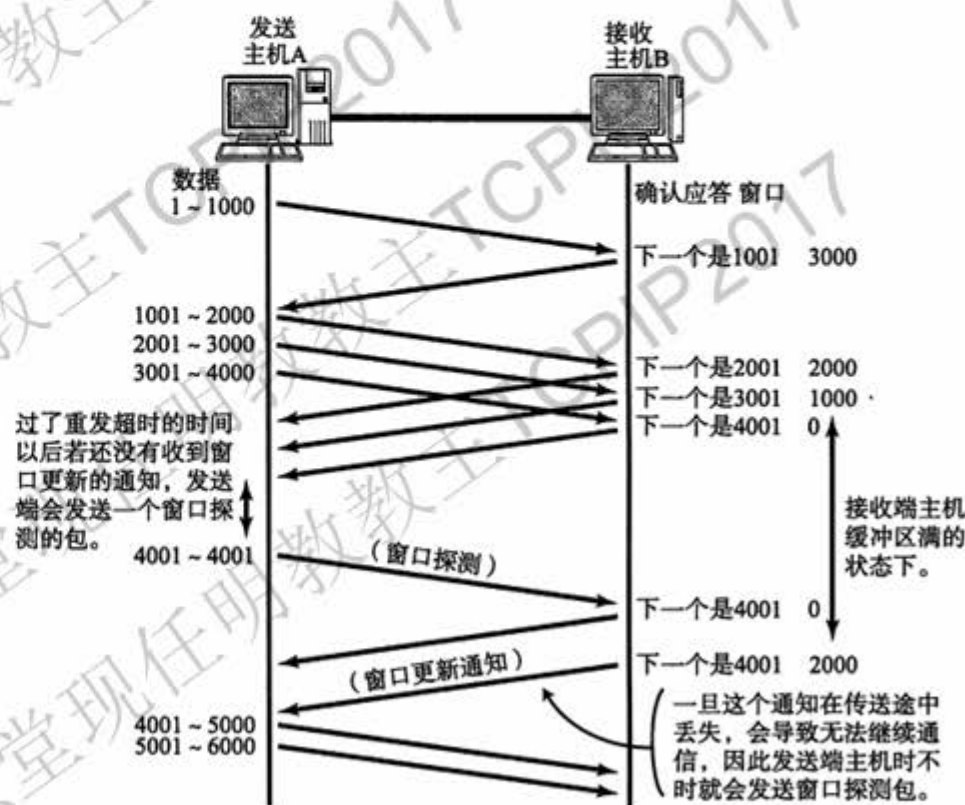
为了防止这种现象的发生，**TCP提供一种机制可以让发送端根据接收端的实际接收能力控制发送的数据量。这就是所谓的流控制。**它的具体操作是，接收端主机向发送端主机通知自己可以接收数据的大小，于是发送端会发送不超过这个限度的数据。**该大小限度就被称作窗口大小。**在上一节中所介绍的**窗口大小的值就是由接收端主机决定的。**

TCP首部中，专门有一个字段用来通知窗口大小。**接收主机将自己可以接收的缓冲区大小放入这个字段中通知给发送端。这个字段的值越大，说明网络的吞吐量越高。**

不过，接收端的这个缓冲区一旦**面临数据溢出时，窗口大小的值也会随之被设置为一个更小的值通知给发送端，从而控制数据发送量。**也就是说，发送端主机会根据接收端主机的指示，对发送数据的量进行控制。这也就形成了一个完整的TCP流控制（流量控制）。

7. 流控制 (2)

如右图所示，当接收端收到从3001号开始的数据段后其缓冲区即满，**不得不暂时停止接收数据（窗口为0）**。之后，在收到发送窗口更新通知后通信才得以继续进行。**如果这个窗口的更新通知在传送途中丢失**，可能会导致无法继续通信。为避免此类问题的发生，发送端主机会时不时的发送一个叫做**窗口探测**的数据段，此数据段仅含一个字节以获取最新的窗口大小信息。



发送端主机根据接收端主机的窗口大小通知进行流量控制。由此也可以防止发送端主机一次发送过大数据导致接收端主机无法处理的情况发生。



8. 拥塞控制（1）

有了TCP的窗口控制，收发主机之间即使不再以一个数据段为单位发送确认应答，也能够连续发送大量数据包。然而，如果在通信刚开始时就发送大量数据，也可能会引发其他问题。

一般来说，计算机网络都处在一个共享的环境。因此也有可能因为其他主机之间的通信使得网络拥堵。在网络出现拥堵时，如果突然发送一个较大量的数据，极有可能会导致整个网络的瘫痪。

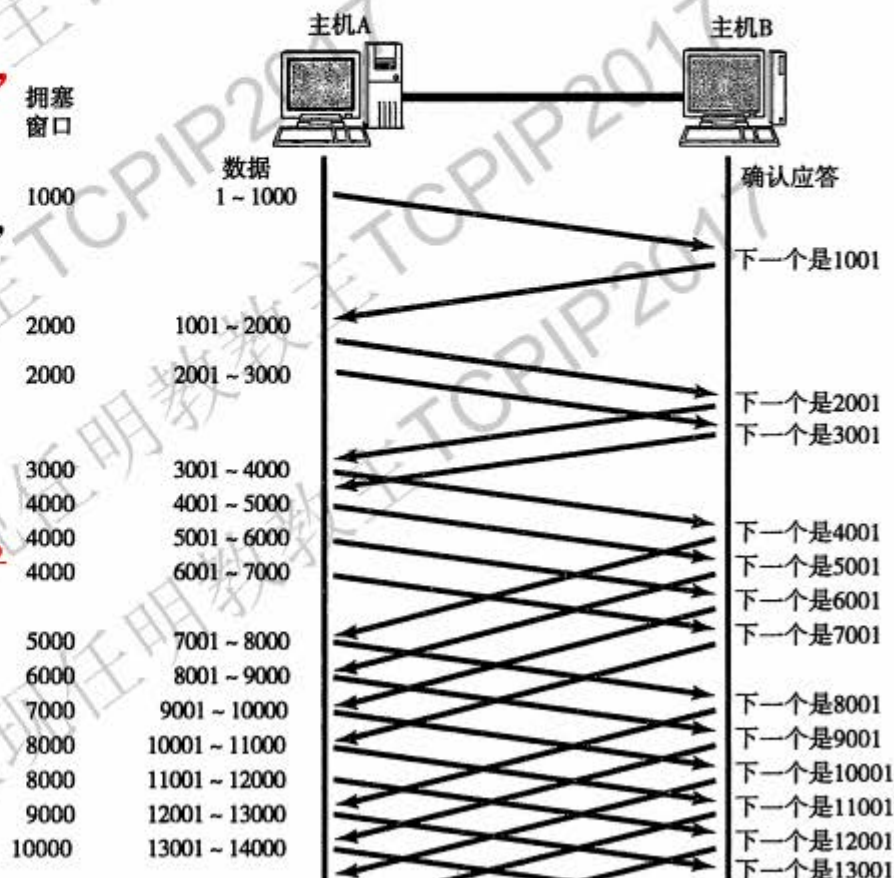
TCP为了防止该问题的出现，在通信一开始时就会通过一个叫做慢启动的算法得出的数值，对发送数据量进行控制。

第五部分.1:TCP协议介绍

慢启动

首先，为了在发送端调节所要发送数据的量，定义了一个叫做“**拥塞窗口**”的概念。于是在**慢启动**的时候，将这个拥塞窗口的大小设置为1个数据段（1MSS），发送数据，**之后每收到一次确认应答（ACK），拥塞窗口的值就加1**。在发送数据包时，**将拥塞窗口的大小与接收端主机通知的窗口大小做比较，然后按照它们当中较小那个值，发送比其还要小的数据量**。

如果重发采用超时机制，那么拥塞窗口的初始值可以设置为1以后再进行慢启动修正。有了上述这些机制，就可以有效地减少通信开始时连续发包，导致的网络拥堵，还可以避免网络拥塞情况的发生。



最初将发送端的窗口（拥塞窗口）设置为1。每收到一个确认应答，窗口的值会增加1个段。（图中所示为没有延迟确认应答的情况，因此与实际情况有所不同）



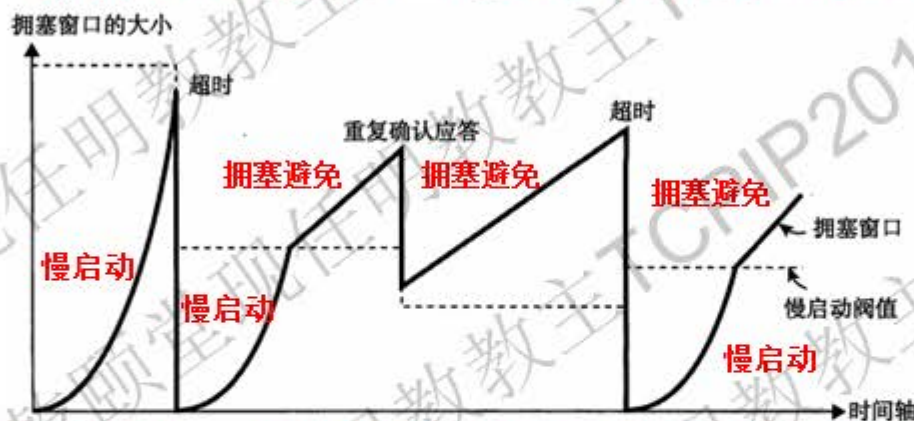
慢启动阈值

不过，随着包的每次往返，拥塞窗口也会以1、2、4等指数函数的增长，拥堵状况激增甚至导致网络拥塞的发生。为了防止这些，引入了慢启动阈值的概念。只要拥塞窗口的值超出这个阈值，在每收到一次确认应答时，只允许以下面这种比例放大拥塞窗口：

$$\frac{1 \text{ 个数据段的字节数}}{\text{拥塞窗口 (字节)}} \times 1 \text{ 个数据段字节数}$$

第五部分.1:TCP协议介绍

拥塞窗口变化示意图



TCP的通信开始时，并没有设置相应的慢启动阈值而是在超时重发时，才会设置为当时拥塞窗口一半的大小。

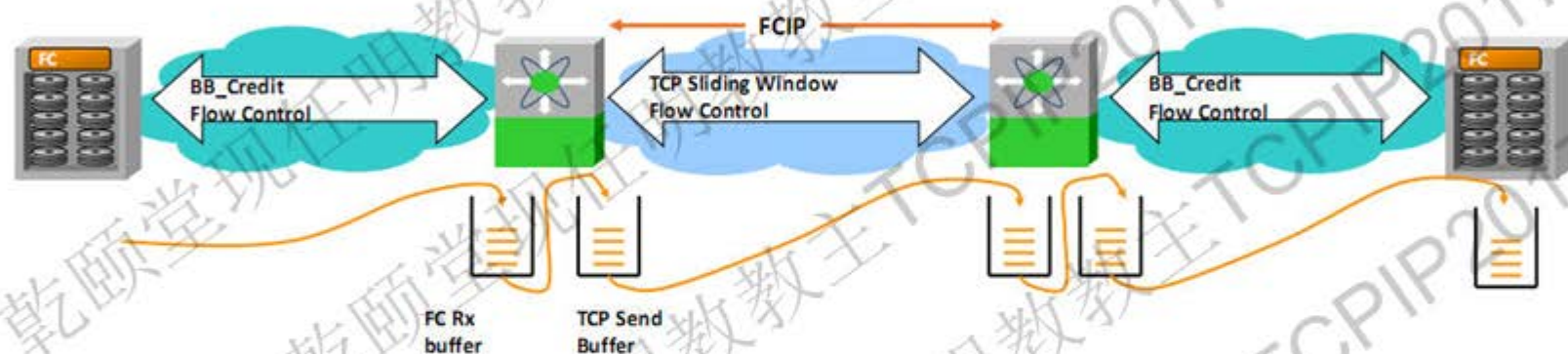
由重复确认应答而触发的高速重发与超时重发机制的处理多少有些不同。因为前者要求至少3次的确认应答数据段到达对方主机后才会触发，相比后者网络的拥堵要轻一些。

而由重复确认应答进行高速重发控制时，慢启动阈值的大小被设置为当时窗口大小的一半。然后将窗口的大小设置为该慢启动阈值+3个数据段的大小。

有了这样一种控制，TCP的拥塞窗口上图所示发生变化。由于窗口的大小会直接影响数据被转发时的吞吐量，所以一般情况下，窗口越大，越会形成高吞吐量的通信。

第五部分.1:TCP协议介绍

关于FCIP



Differences with Normal TCP: (与普通TCP的不同)

- When congestion occurs with other conventional TCP traffic, FCIP is more aggressive during recovery (— **“bullying”**霸占 the other traffic)
---Aggression is proportional to the min-available-bandwidth configuration
激进的程度取决于 **“min-available-bandwidth”** 的配置

Initial rate controlled by — **“min-available-bandwidth”**

Max rate controlled by — **“max-bandwidth”**



9. 提高网络利用率的规范（1）

- **Nagle算法[发送控制]（Cisco激活命令:service nagle）**

TCP中为了提高网络的利用率，经常使用一个叫做Nagle的算法。

该算法是指发送端即使还有应该发送的数据，但如果这部分数据很少的话（解决“**small-packet problem**”问题），则进行延迟发送的一种处理机制。具体来说，就是仅在下列任意一种条件下才能发送数据。如果两个条件都不满足，那么暂时等待一段时间以后再进行数据发送。

1. 已发送的数据都已经收到确认应答时
2. 可以发送最大段长度(MSS)的数据时

根据这个算法虽然网络利用率可以提高，但是可能会发生某种程度的延迟。为此，在窗口系统，以及机械控制等领域中使用TCP时，往往会关闭对该算法的启用。



9. 提高网络利用率的规范（2）

- 延迟确认应答[确认控制]

接收数据的主机如果每次都立刻回复确认应答的话，可能会返回一个较小的窗口。那是因为刚接收完数据，缓冲区已满。

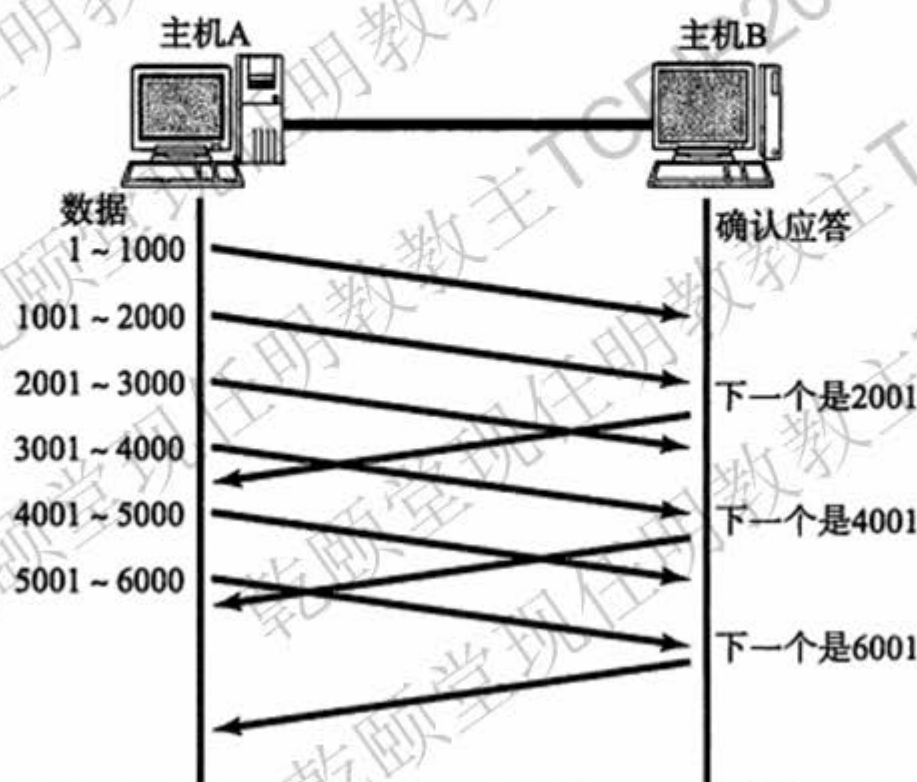
当某个接收端收到这个小窗口的通知以后，会以它为上限发送数据，从而又降低了网络的利用率。为此，引入了一个方法，那就是收到数据以后并不立即返回确认应答，而是延迟一段时间的机制。

- 在没有收到2x最大段长度的数据为止不做确认应答（根据操作系统的不同，有时也有不论数据大小，只要收到两个包就即刻返回确认应答的情况。）
- 其他情况下，最大延迟0.5秒发送确认应答，（很多操作系统设置为0.2秒左右）

事实上，大可不必为每一个数据段都进行一次确认应答。TCP采用滑动窗口的控制机制，因此通常确认应答少一些也无妨。TCP文件传输中，绝大多数是每两个数据段返回一次确认应答。

9. 提高网络利用率的规范 (2)

• 延迟确认应答示意图



每收到两个数据段发送一次的确认应答。不过，等待0.2秒以后没有其他数据包到达的情况下才会发送确认应答。



9. 提高网络利用率的规范（3）

• 捎带应答

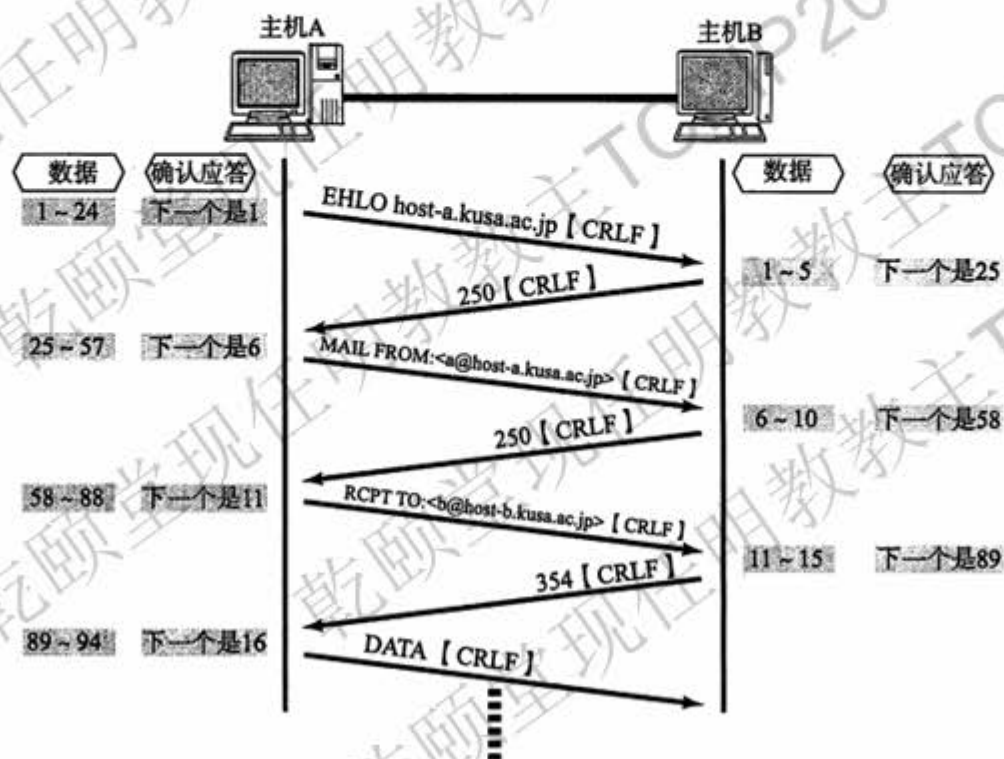
根据应用层协议，发送出去的消息到达对端，对端进行处理以后，会返回一个回执。例如，电子邮件协议的SMTP或POP、文件传输协议中的连接控制部分等。下图所示，**这些应用协议使用同一个连接进行数据的交互**。即使是使用WWW的HTTP协议，从1.1版本以后也是如此。再例如远程登录中针对输入的字符进行回送校验，也是对发送消息的一种回执。

在此类通信当中，TCP的确认应答和回执数据可以通过一个包发送。这种方式叫做捎带应答，(PiggyBack Acknowledgement)。通过这种机制，可以使收发的数据量减少。

另外，接收数据以后如果立刻返回确认应答，就无法实现捎带应答。而是将所接收的数据传给应用处理生成返回数据以后再进行发送请求为止，必须一直等待确认应答的发送。**也就是说，如果没有启用延迟确认应答就无法实现捎带应答。延迟确认应答是能够提高网络利用率从而降低计算机处理负荷的一种较优的处理机制。**

9. 提高网络利用率的规范 (3)

捎带应答



捎带应答是指在同一个TCP包中既发送数据又发送确认应答的一种机制。由此，网络的利用率会提高，计算机的负荷也会减轻。不过，确认应答必须得等到应用处理完数据并将作为回执的数据返回为止，才能进行捎带应答。



第五部分.1:TCP协议介绍

Telnet测试捎带应答

TCP - Transport Control Protocol

- Source Port: 54283 [34-35]
- Destination Port: 23 telnet [36-37]
- Sequence Number: 3964676424 [38-41]
- Ack Number: 3598992022 [42-45]
- TCP Offset: 5 (20 bytes) [46 Mask 0xF0]
- Reserved: %0000 [46 Mask 0x0F]
- TCP Flags: %00011000 ...AP... [47]
- Window: 4054 [48-49]
- TCP Checksum: 0xBBFA [50-51]
- Urgent Pointer: 0 [52-53]
- No TCP Options

TELNET - Network Virtual Terminal

- Line 1: q [54 Mask 0xFFFFFFFF]

Extra bytes

- Number of bytes: (5 bytes) [55-59]

FCS - Frame Check Sequence

- FCS: 0x3496F87C Calculated

TCP - Transport Control Protocol

- Source Port: 23 telnet [34-35]
- Destination Port: 54283 [36-37]
- Sequence Number: 3598992022 [38-41]
- Ack Number: 3964676425 [42-45]
- TCP Offset: 5 (20 bytes) [46 Mask 0xF0]
- Reserved: %0000 [46 Mask 0x0F]
- TCP Flags: %00011000 ...AP... [47]
- Window: 4090 [48-49]
- TCP Checksum: 0xBB05 [50-51]
- Urgent Pointer: 0 [52-53]
- No TCP Options

TELNET - Network Virtual Terminal

- Line 1: q [54 Mask 0xFFFFFFFF]

Extra bytes

- Number of bytes: (5 bytes) [55-59]

FCS - Frame Check Sequence

- FCS: 0xFC42A746 Calculated

- 1.发送确认
- 2.发送回显数据